A VISUAL, MATH-FIRST TUTORIAL

Gradient Descent

From Basic Intuition to Pro-Level Implementation

Batch · Stochastic · Mini-Batch — the math, the code, and the pictures, explained from scratch.

Cost Functions | Partial Derivatives | Batch GD | Stochastic GD | Mini-Batch GD | Multiple Regression | Python From Scratch

TABLE OF CONTENTS

What's Inside

01 What Is Gradient Descent? (The Core Intuition)

02 The Problem: Fitting a Line to Data

03 The Cost Function — Measuring "Wrongness"

04 The Math — Deriving the Gradient from Scratch

05 The Gradient Descent Algorithm, Step by Step

06 The Learning Rate — Too Small, Too Big, Just Right

07 Visualizing the Loss Surface (3D Bowl & Contours)

08 The Three Flavors of Gradient Descent — Overview

09 Batch Gradient Descent — Deep Dive + Code

10 Stochastic Gradient Descent — Deep Dive + Code

11 Mini-Batch Gradient Descent — Deep Dive + Code

12 Head-to-Head Comparison & Convergence Plots

13 Going Pro: Multiple Linear Regression (Vectorized Math)

14 Full Python Implementation — A Reusable Class

15 Pro Tips: Scaling, Learning-Rate Schedules & Pitfalls

16 Cheat Sheet & Summary

LEVEL: BASIC

01 · What Is Gradient Descent?

Gradient Descent is an optimization algorithm. Its entire job is to find the values of some parameters that make a function as small as possible. In machine learning, that function is almost always a **loss function** (also called a cost function) — a number that tells us "how wrong" our model's predictions currently are.

INTUITION — WALKING DOWN A FOGGY MOUNTAIN

Imagine you are standing on a mountain at night, in thick fog. You cannot see the valley below, but you want to reach the lowest point. What do you do? You feel the ground under your feet to sense which direction is **steepest downhill**, take a small step in that direction, and repeat. Do this enough times and you will eventually reach the bottom.

That is exactly what gradient descent does — except the "mountain" is the loss function, your position is the model's current parameters (like slope `m` and intercept `b`), and "feeling the steepest direction" is calculating the **gradient** (a set of partial derivatives).

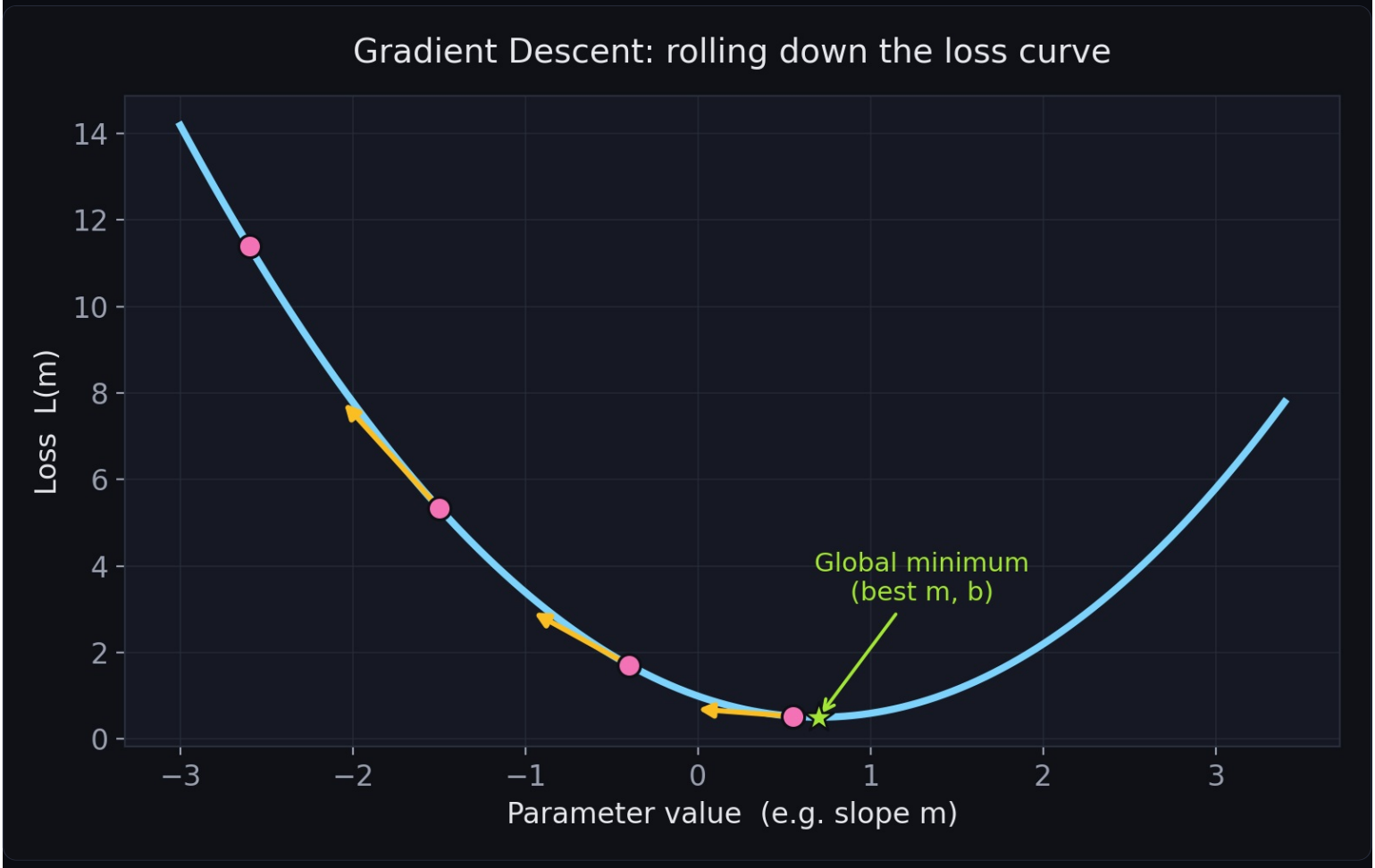


Fig 1.1 — At every point on the curve, the slope (tangent) tells us which direction increases the loss. We move in the **opposite** direction, and the steps naturally get smaller as we approach the bottom because the slope flattens out.

Why do we need this at all?

For very simple problems (like plain linear regression with one input), we could solve for the best parameters directly using algebra — this is called the **Normal Equation** / closed-form solution. But that approach requires inverting a matrix, which becomes extremely slow or even impossible once you have thousands of features or millions of rows of data. Gradient descent instead **iteratively improves** the parameters a little bit at a time, which scales to huge datasets and huge models (including deep neural networks, where no closed-form solution exists at all).

KEY IDEA IN ONE LINE

Start with a guess → measure how wrong you are → find the direction that reduces the wrongness fastest → take a small step that way → repeat until you stop improving.

LEVEL: BASIC

02 · The Problem: Fitting a Line to Data

To make gradient descent concrete, we'll use the same running example throughout this tutorial: **simple linear regression**. Suppose we have data points (for example, CGPA vs. package, or hours studied vs. marks scored) and we want to fit a straight line through them:

$$\hat{y} = m \cdot x + b$$

Here:

- `x` — the input feature (e.g. CGPA)
- `ŷ` ("y-hat") — the model's **predicted** output
- `y` — the **actual** observed output (e.g. real package)
- `m` — the slope of the line (a parameter we need to learn)
- `b` — the intercept of the line (a parameter we need to learn)

Training the model simply means: **find the values of `m` and `b` that make our predictions ŷ as close as possible to the real values y**, across all the data points we have. Gradient descent is one way (a very general and powerful way) to search for those values.

> **BEFORE YOU CONTINUE**
>
> This tutorial assumes you already know what linear regression is trying to do (draw a best-fit line). If any of the terms above feel new, that's fine — we re-derive everything from first principles in the next section, so just keep reading.

LEVEL: BASIC → INTERMEDIATE

03 · The Cost Function

We need one single number that tells us how good or bad our current line (our current `m` and `b`) is. The standard choice for regression is **Mean Squared Error (MSE)**:

$$L(m, b) = (1/n) \cdot \Sigma_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Reading it out loud: *"For every data point, take the difference between the actual value and the predicted value, square it (so negative and positive errors don't cancel out and big errors are punished more), add all of those squared errors up, and divide by the number of points to get an average."*

WHY SQUARE THE ERROR INSTEAD OF JUST USING THE DIFFERENCE?

- Squaring removes the sign, so an error of −5 and +5 both count as "5 units wrong".
- Squaring punishes large errors disproportionately more than small ones, pushing the model to avoid big mistakes.
- Most importantly for us: the squared function is **smooth and differentiable everywhere**, which is exactly what we need for calculus-based optimization like gradient descent.

Substituting $\hat{y}_i = m \cdot x_i + b$, the full cost function becomes:

$$L(m, b) = (1/n) \cdot \Sigma (y_i - (m \cdot x_i + b))^2$$

This is a function of two variables, `m` and `b`. If we plot L(m, b) in 3D (with m and b on the horizontal axes and the loss value on the vertical axis), we always get a nice, smooth **bowl shape** for linear regression — because MSE is a convex function. That bowl shape is exactly why gradient descent works so reliably here: no matter where you start on the bowl, walking downhill always leads you to the single lowest point.

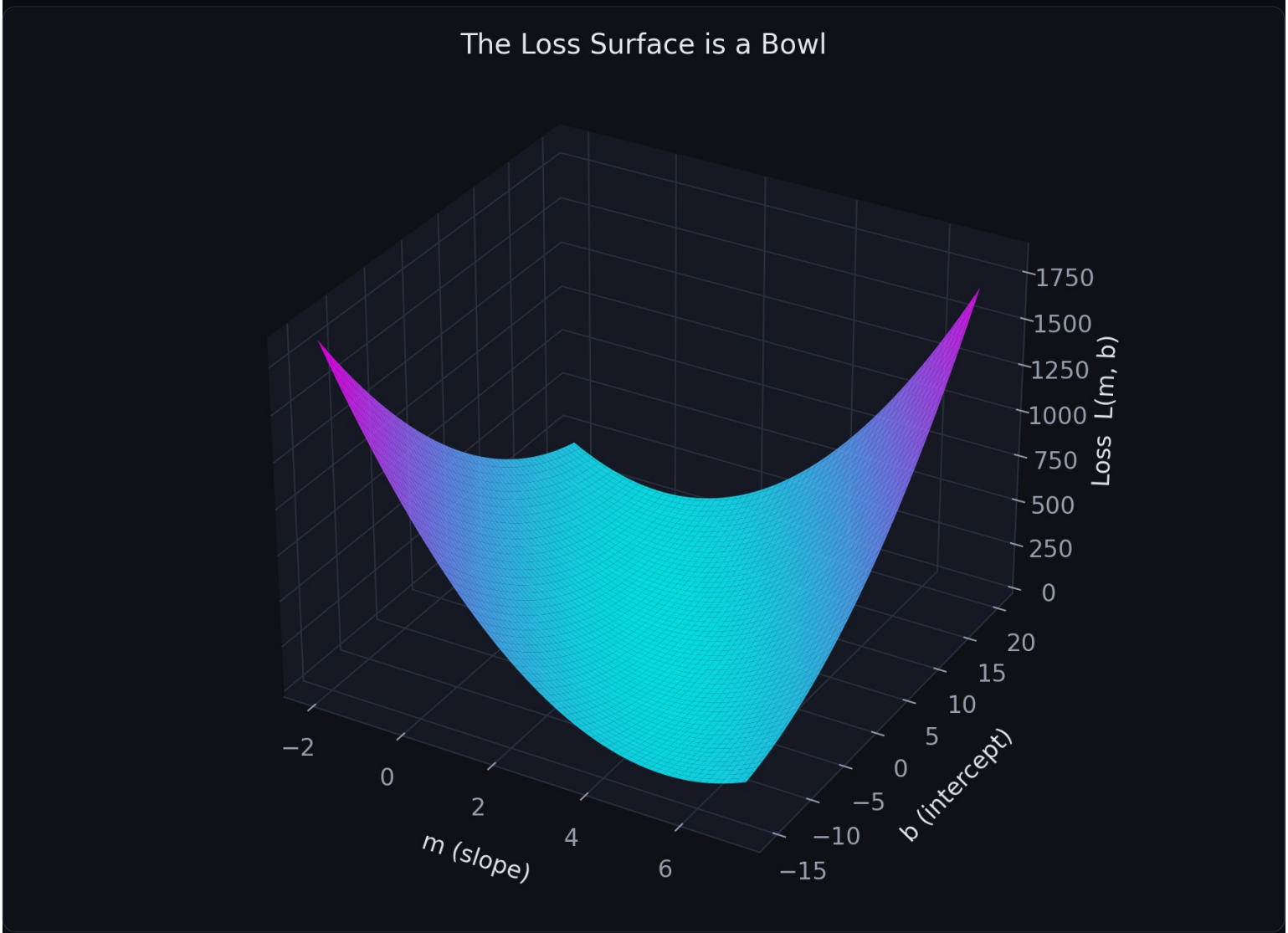


Fig 3.1 — The MSE loss surface for our line-fitting problem, plotted against every combination of slope (m) and intercept (b). Our goal is to find the (m, b) pair sitting at the very bottom of this bowl.

LEVEL: INTERMEDIATE · MATH-HEAVY

04 · Deriving the Gradient From Scratch

To walk downhill, we need to know, at our current position, which direction increases the loss the fastest. That direction is given by the **gradient** — the vector of partial derivatives of the loss with respect to each parameter: ∂L/∂m and ∂L/∂b.

Step 1 — Derivative of the loss w.r.t. the slope (m)

We start from the cost function and apply the chain rule.

CHAIN RULE DERIVATION — ∂L/∂m

$$L(m, b) = (1/n) \Sigma (y_i - m \cdot x_i - b)^2$$

Let $u_i = y_i - m \cdot x_i - b$, so $L = (1/n) \Sigma u_i^2$.

By the chain rule: $\partial L/\partial m = (1/n) \Sigma 2 \cdot u_i \cdot (\partial u_i/\partial m)$

Since $\partial u_i/\partial m = -x_i$, we substitute:

$$\partial L/\partial m = (-2/n) \cdot \Sigma x_i \cdot (y_i - \hat{y}_i)$$

Step 2 — Derivative of the loss w.r.t. the intercept (b)

CHAIN RULE DERIVATION — ∂L/∂b

Same starting point, but now $\partial u_i/\partial b = -1$ (b has no coefficient attached):

$$\partial L/\partial b = (-2/n) \cdot \Sigma (y_i - \hat{y}_i)$$

These two formulas are the entire mathematical engine of gradient descent for simple linear regression. Notice the pattern: both derivatives are proportional to the **error** $(y_i - \hat{y}_i)$; the slope's derivative additionally weights each error by that point's x-value, because x_i is "attached" to m in the original equation.

Step 3 — The parameter update rule

Once we know which direction increases the loss (the gradient), we move a small step in the **opposite** direction — hence "descent". The step size is controlled by a hyperparameter α (alpha), called the **learning rate**:

$$m \leftarrow m - \alpha \cdot \partial L/\partial m \qquad b \leftarrow b - \alpha \cdot \partial L/\partial b$$

We repeat this update again and again — each pass is called an **iteration** or, when it covers the whole dataset once, an **epoch** — until the loss stops decreasing meaningfully (this is called **convergence**).

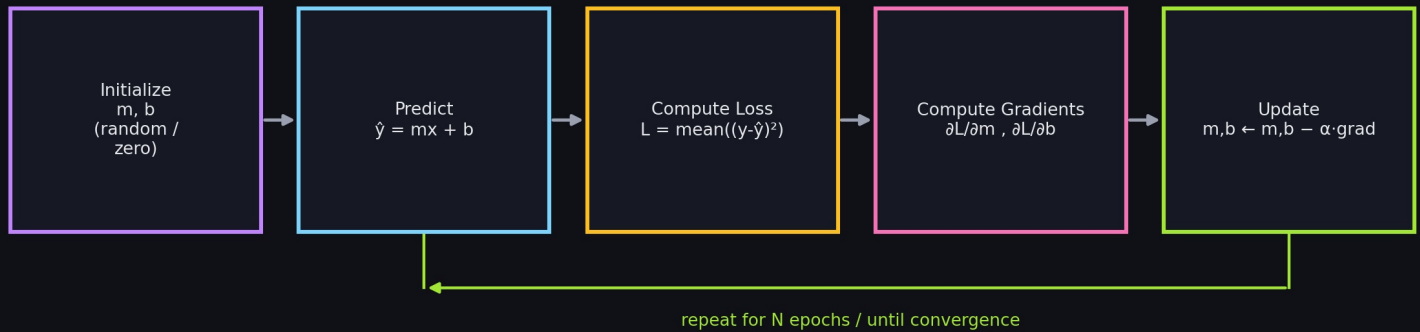


Fig 4.1 — One full loop of the gradient descent algorithm. This block repeats for a fixed number of epochs, or until the loss barely changes between iterations.

LEVEL: INTERMEDIATE

05 · The Algorithm, Step by Step

Putting everything together, here is the full gradient descent algorithm for simple linear regression:

PSEUDOCODE

```
initialize m = 0, b = 0           # or small random values
set learning_rate = α, epochs = N

repeat N times:
    ŷ = m * x + b                 # step 1: predict
    error = y - ŷ                 # step 2: how wrong are we?
    dL_dm = (-2/n) * sum(x * error)  # step 3: gradient wrt m
    dL_db = (-2/n) * sum(error)      # step 3: gradient wrt b
    m = m - α * dL_dm             # step 4: update m
    b = b - α * dL_db             # step 4: update b

return m, b
```

Every one of the "three types" of gradient descent you'll learn in this guide (Batch, Stochastic, Mini-batch) uses **exactly this update rule**. The only thing that changes between them is **how much data** we use to compute the error / gradient in each single update. Keep that one sentence in your head — it is the entire secret behind the three variants.

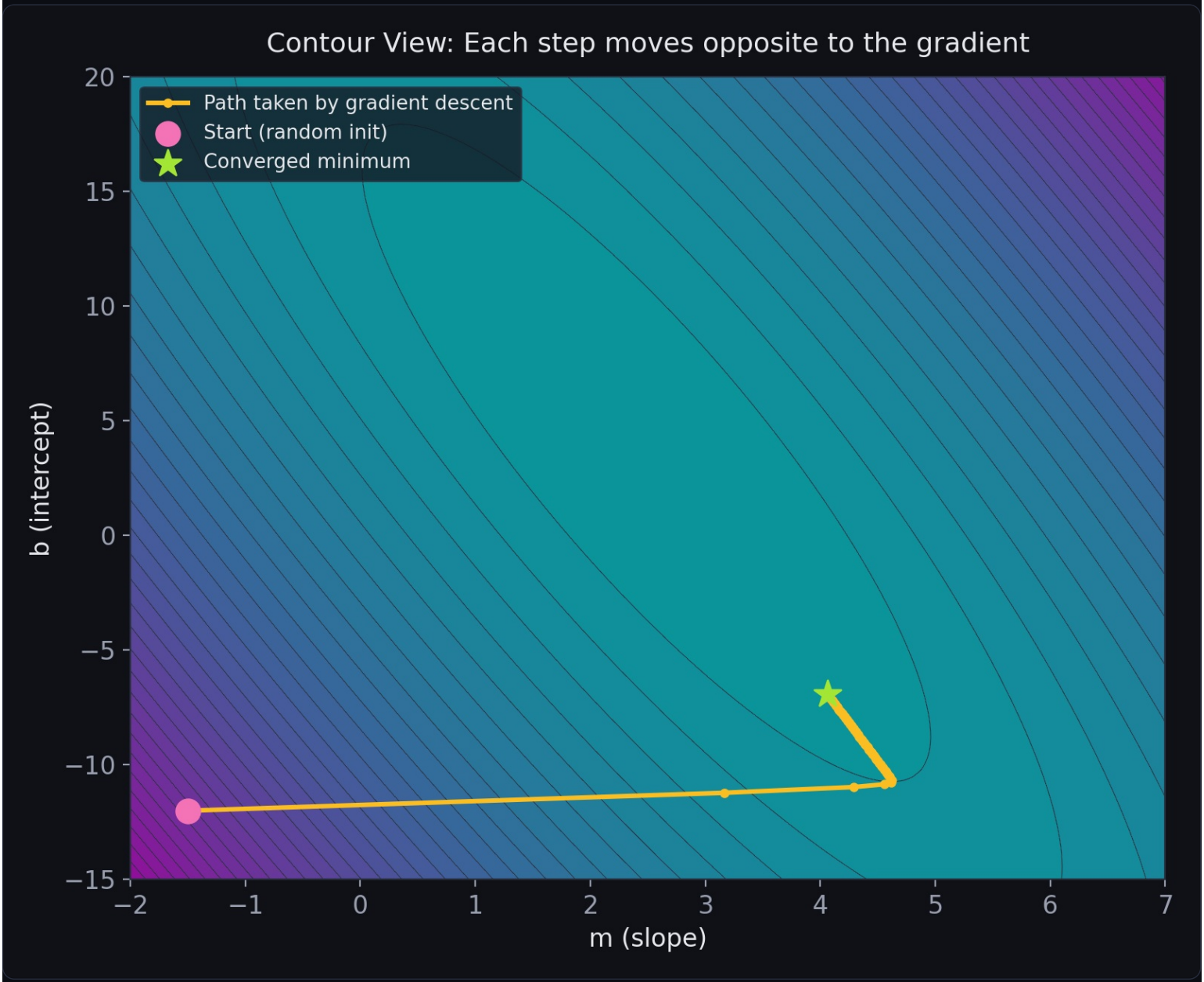


Fig 5.1 — Looking at the bowl from directly above (a contour / top-down view). Each dot is one iteration. Notice the path always moves perpendicular to the contour lines — i.e. directly along the steepest downhill direction — and the steps shrink automatically as the surface flattens near the minimum.

LEVEL: INTERMEDIATE

06 · The Learning Rate (α)

The learning rate is arguably the single most important hyperparameter in gradient descent. It controls how big a step we take at every iteration.

TOO SMALL

Training is extremely slow — it may take thousands of extra iterations to converge, wasting time and compute.

JUST RIGHT

The loss decreases smoothly and quickly settles near the minimum in a reasonable number of steps.

TOO LARGE

Steps overshoot the minimum, bouncing back and forth — the loss can oscillate wildly or even diverge to infinity.

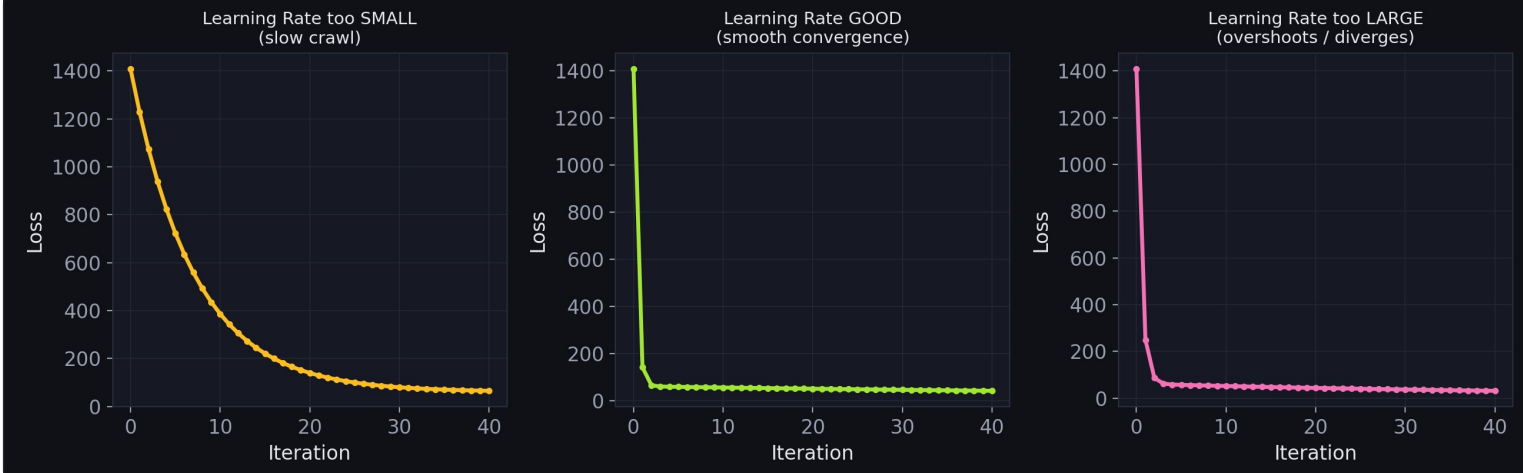


Fig 6.1 — The same starting point and the same algorithm, run with three different learning rates. Left: barely moves after 40 iterations. Middle: converges cleanly. Right: the loss explodes as the steps overshoot the bowl walls again and again.

PRACTICAL ADVICE

- Common starting points to try: `0.1, 0.01, 0.001, 0.0001` — then narrow down.
- If the loss is decreasing but painfully slowly → increase α.
- If the loss is jumping around erratically or increasing → decrease α.
- Advanced optimizers (Adam, RMSProp, momentum-based SGD) adapt the learning rate automatically — more on this in the Pro Tips section.

LEVEL: INTERMEDIATE → ADVANCED

08 · The Three Flavors of Gradient Descent

Recall from Section 5: the update rule never changes. What changes is **how many data points we look at before making one update** to m and b. This single design choice gives us three named variants:



Fig 8.1 — All three variants sit on one spectrum, defined purely by batch size.

Variant	Data used per update	Updates per epoch	Path to minimum
BATCH GD	All `n` rows	1	Smooth, direct
STOCHASTIC GD	1 row	`n`	Noisy, zig-zag
MINI-BATCH GD	A small batch (e.g. 16–256 rows)	`n / batch_size`	Balanced

Over the next three sections we will study each one in depth: the intuition, the exact math change (spoiler: it's tiny), the trade-offs, and working Python code for each.

LEVEL: ADVANCED

09 · Batch Gradient Descent BATCH

This is exactly the algorithm we derived in Sections 4–5. It is called **Batch** Gradient Descent because every single update uses the **entire batch** of training data at once.

$$\partial L/\partial m = (-2/n) \cdot \Sigma_{i=1}^{n} x_i(y_i - \hat{y}_i) \qquad \partial L/\partial b = (-2/n) \cdot \Sigma_{i=1}^{n} (y_i - \hat{y}_i)$$

The sum runs over **all n rows** before a single update to m or b happens.

ADVANTAGES

- Stable, smooth convergence — the gradient is the true average direction, so no noise.
- Converges reliably to the minimum for convex problems like linear regression.
- Easy to reason about and debug.

DISADVANTAGES

- Extremely slow on large datasets — every single step requires scanning the whole dataset.
- High memory usage if the dataset doesn't fit in memory.
- Cannot be used for online / streaming learning (new data arriving continuously).

PYTHON — BATCH GRADIENT DESCENT FROM SCRATCH

```python
class BatchGDRegressor:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.m = 0
        self.b = 0
        self.lr = learning_rate
        self.epochs = epochs

    def fit(self, X, y):
        n = len(X)
        for _ in range(self.epochs):
            y_pred = self.m * X + self.b           # predict on ALL rows
            error = y - y_pred
            dm = (-2/n) * sum(X * error)          # gradient uses ALL rows
            db = (-2/n) * sum(error)
            self.m = self.m - self.lr * dm
            self.b = self.b - self.lr * db
        return self.m, self.b

    def predict(self, X):
        return self.m * X + self.b
```

WHEN TO USE IT

Small-to-medium datasets (roughly, up to a few tens of thousands of rows) where you can afford to scan all the data on every iteration, and you want the most stable, predictable convergence.

LEVEL: ADVANCED

10 · Stochastic Gradient Descent SGD

"Stochastic" simply means *random*. Instead of using all n rows to compute one update, Stochastic Gradient Descent picks **a single random data point** and updates the parameters based on the error at just that one point. It then moves to the next (usually shuffled) point, and so on — so a single epoch now produces `n` updates instead of just 1.

$$\partial L/\partial m \approx -2 \cdot x_i(y_i - \hat{y}_i) \qquad \partial L/\partial b \approx -2 \cdot (y_i - \hat{y}_i)$$

Notice: the summation and the (1/n) averaging are gone entirely — we compute the "gradient" using just one (x_i, y_i) pair. This is only an **estimate** of the true gradient (which is why the path is noisy), but on average, across many random points, it still points roughly downhill.

ADVANTAGES

- Very fast updates — no need to scan the whole dataset before improving.
- Low memory footprint — perfect for huge or streaming datasets.
- The noise can actually help escape shallow local minima in non-convex problems (e.g. neural networks).

DISADVANTAGES

- Very noisy convergence — the loss bounces around instead of decreasing smoothly.
- May never settle exactly at the minimum; it "dances" around it.
- Harder to parallelize / vectorize efficiently (one row at a time).

PYTHON — STOCHASTIC GRADIENT DESCENT FROM SCRATCH

```python
import random

class SGDRegressor:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.m = 0
        self.b = 0
        self.lr = learning_rate
        self.epochs = epochs

    def fit(self, X, y):
        n = len(X)
        for _ in range(self.epochs):
            for _ in range(n):                      # n updates per epoch
                idx = random.randint(0, n - 1)      # pick ONE random row
                xi, yi = X[idx], y[idx]
                y_pred = self.m * xi + self.b
                error = yi - y_pred
                dm = -2 * xi * error             # no averaging, no sum
                db = -2 * error
                self.m = self.m - self.lr * dm
                self.b = self.b - self.lr * db
        return self.m, self.b

    def predict(self, X):
        return self.m * X + self.b
```

WHEN TO USE IT

Very large or streaming datasets where you can't afford to wait for a full pass before improving, or online learning systems that must adapt continuously to new incoming data.

LEVEL: ADVANCED

11 · Mini-Batch Gradient Descent MINI-BATCH

Mini-Batch Gradient Descent is the practical compromise the industry actually uses most of the time (it's the default in almost every deep learning framework). Instead of one row (SGD) or all rows (Batch), we split the data into small random chunks — **batches** — of, say, 16, 32, 64, or 256 rows, and perform one update per batch.

$$\partial L/\partial m = (-2/k) \cdot \Sigma_{i \in batch}\, x_i(y_i - \hat{y}_i) \qquad \partial L/\partial b = (-2/k) \cdot \Sigma_{i \in batch}\, (y_i - \hat{y}_i)$$

where `k` is the batch size. Set k = 1 and this formula becomes SGD; set k = n (the whole dataset) and it becomes Batch GD. Mini-batch really is the middle ground on the very same spectrum.

ADVANTAGES

- Much faster than Batch GD, much smoother than SGD.
- Can exploit vectorized / GPU matrix operations on each batch for real speed.
- Works well with huge datasets that don't fit in memory all at once.

DISADVANTAGES

- One more hyperparameter to tune: the batch size.
- Still somewhat noisy compared to full Batch GD (though far less than SGD).

PYTHON — MINI-BATCH GRADIENT DESCENT FROM SCRATCH

```python
import numpy as np

class MiniBatchGDRegressor:
    def __init__(self, learning_rate=0.01, epochs=100, batch_size=32):
        self.m = 0
        self.b = 0
        self.lr = learning_rate
        self.epochs = epochs
        self.batch_size = batch_size

    def fit(self, X, y):
        n = len(X)
        for _ in range(self.epochs):
            indices = np.random.permutation(n)          # shuffle every epoch
            for start in range(0, n, self.batch_size):
                batch_idx = indices[start:start + self.batch_size]
                xb, yb = X[batch_idx], y[batch_idx]
                k = len(xb)
                y_pred = self.m * xb + self.b
                error = yb - y_pred
                dm = (-2/k) * np.sum(xb * error)   # averaged over the batch
                db = (-2/k) * np.sum(error)
                self.m = self.m - self.lr * dm
                self.b = self.b - self.lr * db
        return self.m, self.b

    def predict(self, X):
        return self.m * X + self.b
```

WHEN TO USE IT

The default choice for most real-world machine learning and deep learning — it balances speed, stability, and hardware efficiency. Typical batch sizes: 16, 32, 64, 128, 256.

LEVEL: ADVANCED

12 · Head-to-Head Comparison

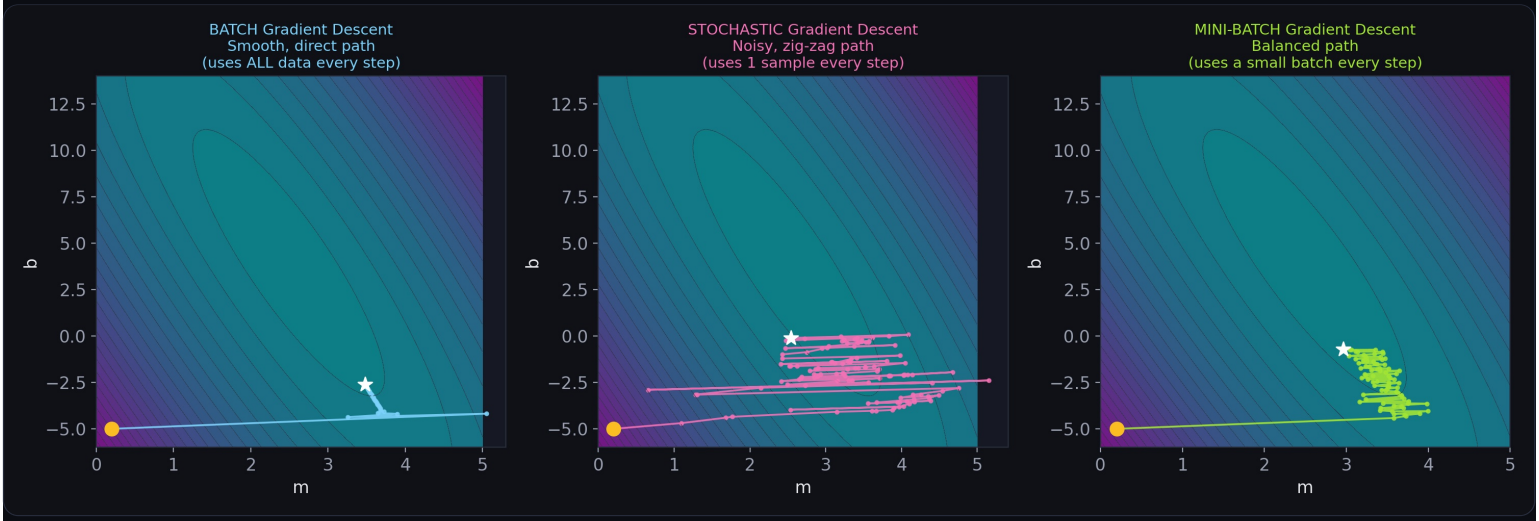


Fig 12.1 — The exact same starting point and (roughly) the same number of total data passes, run with each strategy. Batch GD takes a direct, smooth route. SGD zig-zags aggressively because each step only "sees" one point. Mini-batch strikes a clear middle ground.

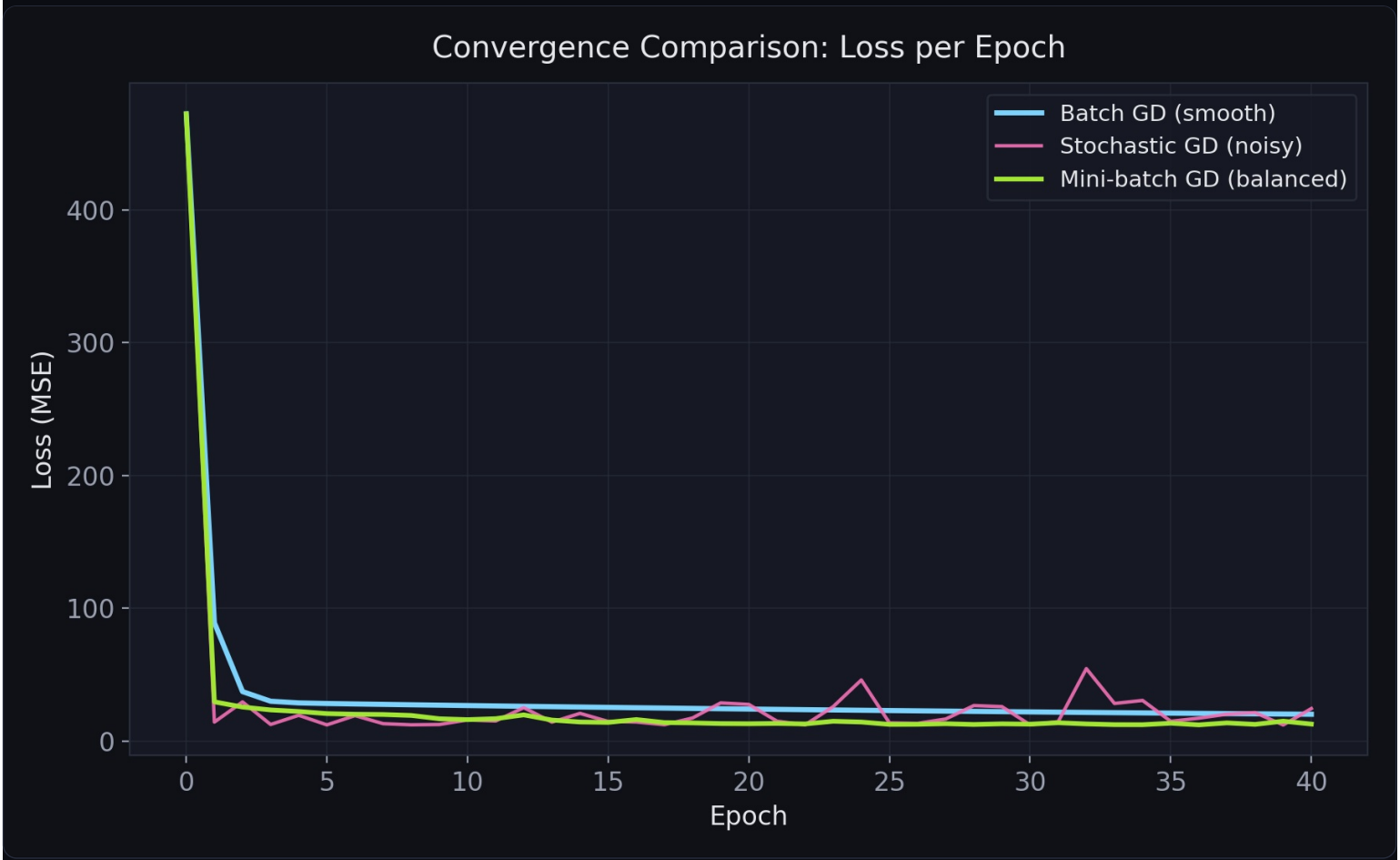


Fig 12.2 — Loss (MSE) plotted against epoch number for all three variants on the same dataset. Batch GD (cyan) is the smoothest. Stochastic GD (pink) is visibly noisier — notice the small jitters even after mostly converging. Mini-batch (lime) converges almost as fast as SGD while staying much smoother.

Criterion	BATCH	STOCHASTIC	MINI-BATCH
Speed per epoch	Slowest	Fast	Fast
Convergence stability	Very stable	Noisy	Moderately stable
Memory usage	High	Very low	Low–Medium
Hardware (GPU/vectorization) friendly	Yes	No	Yes (best fit)
Good for online / streaming data	No	Yes	Somewhat
Common use case	Small/medium classical ML	Very large / streaming data	Deep learning, most modern ML

RULE OF THUMB

If you're not sure which to pick: start with **Mini-Batch Gradient Descent** (batch size 32 or 64). It is the industry-standard default for good reason — it almost always gives you the best trade-off between speed and stability.

LEVEL: PRO

13 · Going Pro: Multiple Linear Regression

Real-world datasets almost never have just one input feature. If we predict a student's package using CGPA, IQ, and gender, we now have **multiple** inputs, and our line becomes a **hyperplane**:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Here β_0 plays the role of the old intercept `b`, and $\beta_1 \ldots \beta_n$ are the "slopes" (weights) for each of the n input columns. If a dataset has n input columns, we now have **n + 1 parameters** to learn (n weights + 1 intercept) instead of just 2.

The loss function, generalized

$$L(\beta) = (1/n) \cdot \Sigma_{i=1}^{n} (y_i - \hat{y}_i)^2$$

The formula looks identical to before — only $\hat{y}_i$ is now computed from many inputs instead of one.

The problem: one derivative per parameter

If we had to compute a partial derivative for every β individually (β_0, β_1, …, β_n), that would mean **n + 1 separate derivative formulas** to track by hand. For a dataset with, say, 50 input columns, that's 51 separate equations. This does not scale. The fix is to switch to **vector / matrix notation**, so that all n + 1 derivatives collapse into a single equation.

SETTING UP THE VECTORIZED FORM

Stack all the data into a matrix `X` of shape (n rows, m+1 columns) — the extra column is a column of all 1's, so the intercept β_0 can be treated just like every other weight. Stack all the parameters into a single column vector β of shape (m+1, 1):

$$\hat{y} = X \cdot \beta \quad \text{(a single matrix multiplication predicts every row at once)}$$

LEVEL: PRO

13 · Vectorized Gradient — Full Derivation

For any single coefficient β_j, following the exact same chain-rule logic from Section 4 (just with x_{ij} — the j-th feature of the i-th row — standing in for the old single x_i), we get:

$$\partial L/\partial \beta_j = (-2/n) \cdot \Sigma_{i=1}^{n} x_{ij}(y_i - \hat{y}_i)$$

This says: *"to update any one weight, multiply the error at every row by that row's value in THAT weight's column, and sum it up."* That is precisely what a **matrix-vector dot product** computes for us, for every weight simultaneously:

THE PUNCHLINE — ONE EQUATION, ALL GRADIENTS

$$\nabla L(\beta) = (-2/n) \cdot X^T \cdot (y - \hat{y})$$

X^T (X transposed) has shape (m+1, n). $(y - \hat{y})$ has shape (n, 1). Their product has shape (m+1, 1) — exactly one gradient value per parameter, computed in a **single line of code**, regardless of whether you have 2 features or 2,000.

And the update rule generalizes just as cleanly:

$$\beta \leftarrow \beta - \alpha \cdot \nabla L(\beta)$$

WHY THIS MATTERS

This is exactly how libraries like Scikit-learn, TensorFlow, and PyTorch implement gradient descent under the hood — as matrix operations, not manual loops over each coefficient. Understanding this derivation is the bridge between "toy" gradient descent and real, production-grade machine learning code.

LEVEL: PRO

14 · Full Python Implementation

Let's put it all together into one reusable, vectorized class that supports multiple input columns and can run as Batch, Stochastic, or Mini-Batch depending on the `batch_size` you pass in.

PYTHON — A COMPLETE, CONFIGURABLE GRADIENT DESCENT REGRESSOR

```python
import numpy as np

class GDRegressor:
    """
    Unified Gradient Descent Linear Regression.
    batch_size = 1        -> Stochastic Gradient Descent
    batch_size = 16..256  -> Mini-Batch Gradient Descent
    batch_size = len(X)   -> Batch Gradient Descent
    """
    def __init__(self, learning_rate=0.01, epochs=100, batch_size=None):
        self.lr = learning_rate
        self.epochs = epochs
        self.batch_size = batch_size
        self.intercept_ = 0.0
        self.coef_ = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.coef_ = np.zeros(n_features)             # beta_1 ... beta_n
        batch_size = self.batch_size or n_samples    # default = full batch

        for _ in range(self.epochs):
            indices = np.random.permutation(n_samples)  # shuffle every epoch
            for start in range(0, n_samples, batch_size):
                idx = indices[start:start + batch_size]
                X_batch, y_batch = X[idx], y[idx]
                k = len(X_batch)

                y_pred = X_batch @ self.coef_ + self.intercept_
                error = y_batch - y_pred

                # vectorized gradients -- works for ANY number of features
                d_coef = (-2/k) * (X_batch.T @ error)
                d_intercept = (-2/k) * np.sum(error)

                self.coef_ -= self.lr * d_coef
                self.intercept_ -= self.lr * d_intercept
        return self

    def predict(self, X):
        return X @ self.coef_ + self.intercept_


# ---- usage ----
# Batch GD:
model = GDRegressor(learning_rate=0.01, epochs=200, batch_size=None)

# Stochastic GD:
model = GDRegressor(learning_rate=0.01, epochs=50, batch_size=1)

# Mini-Batch GD:
model = GDRegressor(learning_rate=0.01, epochs=50, batch_size=32)

model.fit(X_train, y_train)
predictions = model.predict(X_test)
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```

WHAT CHANGED FROM THE SIMPLE VERSION?

- `self.coef_` is now an **array** of weights (one per feature) instead of a single number `m`.
- Predictions use a matrix multiplication `X @ coef_` instead of `m * x`.
- The gradient for all coefficients is computed in one line: `X_batch.T @ error` — this is the exact vectorized formula we derived in Section 13.
- One class, one `batch_size` parameter, all three algorithms.

LEVEL: PRO

15 · Pro Tips, Pitfalls & What's Next

1. Feature scaling matters — a lot

If one input column ranges from 0–1 and another ranges from 0–100,000, the loss surface becomes a long, narrow, elongated bowl instead of a nice round one. Gradient descent then zig-zags badly and converges very slowly. **Standardize** (subtract mean, divide by standard deviation) or **normalize** (scale to 0–1) your features before training.

2. Choosing the number of epochs

Too few epochs and the model hasn't converged yet (underfit training). Too many, and you waste compute with no further improvement (and, combined with a very flexible model, risk overfitting). In practice, plot the loss curve and stop once it visibly flattens out — this is sometimes automated with **early stopping**.

3. Learning rate schedules

Instead of a single fixed α for the whole training run, many practitioners *decay* the learning rate over time — start with a larger α for fast early progress, then shrink it so later steps are finer and don't overshoot near the minimum. Common schedules: step decay, exponential decay, and 1/t decay.

4. Momentum & adaptive optimizers (a preview)

Plain gradient descent (what we built here) is often called "vanilla" gradient descent. Modern deep learning almost always uses smarter variants built on the exact same core idea:

- **Momentum** — accumulates a running average of past gradients so the update "remembers" its direction, smoothing out zig-zags (especially helpful for SGD).
- **RMSProp / Adagrad** — automatically scale the learning rate per-parameter based on how large recent gradients have been.
- **Adam** — combines momentum and adaptive per-parameter learning rates; the most widely used optimizer in deep learning today.

Every one of these still boils down to the same core loop from Section 5 — they just make the update rule smarter.

5. When NOT to use gradient descent

USE THE NORMAL EQUATION INSTEAD WHEN...

Your dataset is small enough (roughly, a few thousand rows and features) that computing $(X^T X)^{-1} X^T y$ directly is fast. It gives the exact optimal answer in one shot with no learning rate to tune — but it doesn't scale to large feature counts (matrix inversion is expensive) and doesn't generalize to non-linear models like neural networks.

LEVEL: ALL

16 · Cheat Sheet & Summary

CORE FORMULAS AT A GLANCE

Simple linear regression cost:

$$L(m,b) = (1/n)\ \Sigma\ (y_i - (mx_i+b))^2$$

Gradients:

$$\partial L/\partial m = (-2/n)\Sigma x_i(y_i-\hat{y}_i) \quad \partial L/\partial b = (-2/n)\Sigma(y_i-\hat{y}_i)$$

Update rule:

$$m \leftarrow m - \alpha\cdot\partial L/\partial m \quad b \leftarrow b - \alpha\cdot\partial L/\partial b$$

Vectorized (multiple regression):

$$\nabla L(\beta) = (-2/n)\cdot X^T(y - \hat{y}) \quad \beta \leftarrow \beta - \alpha\cdot\nabla L(\beta)$$

Term	Meaning
Loss / Cost function	A number measuring how wrong the model's predictions are (we use MSE)
Gradient	The vector of partial derivatives; points in the direction of steepest increase
Learning rate (α)	How big a step to take at each update
Epoch	One full pass through the training dataset
Batch size	Number of samples used to compute one gradient update
Convergence	The point where the loss stops meaningfully decreasing

The Big Picture, One More Time

1. Define a cost function that measures how wrong your model is.
2. Derive the gradient (the partial derivatives of the cost w.r.t. every parameter).
3. Repeatedly nudge every parameter a small step opposite to its gradient.
4. Choose how much data feeds each nudge: all of it (**Batch**), one point (**Stochastic**), or a small chunk (**Mini-Batch**).
5. Tune the learning rate and epoch count until the loss curve flattens out nicely.

YOU NOW KNOW...

How to derive gradient descent from first principles, why the loss surface is a bowl, how the three variants differ mathematically and practically, how to vectorize the math for real datasets with many features, and how to implement all of it in clean, reusable Python code. That's the complete journey from basic to pro. 🎯